Introduction

Behavioral design patterns focus on how objects interact and communicate with each other, defining the flow of control in a system. These patterns make it easier to manage the interactions between objects, promoting loose coupling and enhancing flexibility.

Imagine a remote control that sends commands to various devices, like turning on the lights or adjusting the volume. The user doesn't need to understand the internal workings of the devices, just the commands they can give. This is a perfect example of what behavioral patterns like the **Command Pattern** help us achieve.

The Command Pattern encapsulates a request as an object, allowing for more flexible and dynamic command handling. In the upcoming sections, we'll dive deeper into how the Command Pattern works and how it can be applied in real-world scenarios.

Command Pattern

The **Command Pattern** is a behavioral design pattern that turns a request into a separate object, allowing you to decouple the code that issues the request from the code that performs it.

Formal Definition

The Command Pattern is a behavioral design pattern that **encapsulates a request as an object**, allowing for parameterization of clients with different requests, queuing of requests, and logging of the requests. It **lets you add features like undo, redo, logging, and dynamic command execution** without changing the core business logic.

This allows you to execute commands at a later time, in a flexible manner, without having to interact directly with the request's execution details.

Real-Life Analogy

Think of a **remote control** used to turn on or off the lights or an air conditioner (AC). When you press a button to turn on the lights or adjust the temperature, you don’t need to understand how the internal circuits work or how the AC receives the signal. You just press the **"On"** or **"Off"** button, and the remote control takes care of sending the command.

Similarly, the Command Pattern decouples the sender of a request (the remote control) from the receiver (the light or AC), providing flexibility and simplicity in handling commands.

Four Key Components

- **Client** – Initiates the request and sets up the command object.
- **Invoker** – Asks the command to execute the request.
- **Command** – Defines a binding between a receiver object and an action.
- **Receiver** – Knows how to perform the actions to satisfy a request.

Understanding the Problem

Let's say we're building a simple **remote control system** where devices like lights and air conditioner can be turned on and off. Here's a naive implementation of the code:

```
#include <iostream>

#include <string>

using namespace std;
```

```
// Receiver classes - Light and AC with basic on/off methods
class Light {
public:
    void on() {
        cout << "Light turned ON" << endl;
    }

    void off() {
        cout << "Light turned OFF" << endl;
    }
};

class AC {
public:
    void on() {
        cout << "AC turned ON" << endl;
    }

    void off() {
        cout << "AC turned OFF" << endl;
```

```
    }
};

// Invoker - NaiveRemoteControl class to control devices
class NaiveRemoteControl {
private:
    Light* light;
    AC* ac;
    string lastAction = "";

public:
    NaiveRemoteControl(Light* light, AC* ac) {
        this->light = light;
        this->ac = ac;
    }

    // Command methods
    void pressLightOn() {
        light->on();
        lastAction = "LIGHT_ON";
    }
```

```
void pressLightOff() {
    light->off();
    lastAction = "LIGHT_OFF";
}

void pressACOn() {
    ac->on();
    lastAction = "AC_ON";
}

void pressACOff() {
    ac->off();
    lastAction = "AC_OFF";
}

// Undo last action
void pressUndo() {
    if (lastAction == "LIGHT_ON") {
        light->off();
        lastAction = "LIGHT_OFF";
```

```
        }
        else if (lastAction == "LIGHT_OFF") {
            light->on();
            lastAction = "LIGHT_ON";
        }
        else if (lastAction == "AC_ON") {
            ac->off();
            lastAction = "AC_OFF";
        }
        else if (lastAction == "AC_OFF") {
            ac->on();
            lastAction = "AC_ON";
        }
        else {
            cout << "No action to undo." << endl;
        }
    }
};

// Client Code
int main() {
```

```
    Light light;
    AC ac;
    NaiveRemoteControl remote(&light, &ac);

    remote.pressLightOn();
    remote.pressACOn();
    remote.pressLightOff();
    remote.pressUndo(); // Should undo LIGHT_OFF -> Light ON
    remote.pressUndo(); // Should undo AC_ON -> AC OFF

    return 0;
}

import java.util.*;

// Receiver classes - Light and AC with basic on/off methods
class Light {
    public void on() {
        System.out.println("Light turned ON");
    }
```

```
    public void off() {
        System.out.println("Light turned OFF");
    }
}

class AC {
    public void on() {
        System.out.println("AC turned ON");
    }

    public void off() {
        System.out.println("AC turned OFF");
    }
}

// Invoker - NaiveRemoteControl class to control devices
class NaiveRemoteControl {
    private Light light;
    private AC ac;
    private String lastAction = "";
```

```
public NaiveRemoteControl(Light light, AC ac) {
    this.light = light;
    this.ac = ac;
}

// Command methods
public void pressLightOn() {
    light.on();
    lastAction = "LIGHT_ON";
}

public void pressLightOff() {
    light.off();
    lastAction = "LIGHT_OFF";
}

public void pressACOn() {
    ac.on();
    lastAction = "AC_ON";
}
```

```
    public void pressACOff() {
        ac.off();
        lastAction = "AC_OFF";
    }

    // Undo last action
    public void pressUndo() {
        switch (lastAction) {
            case "LIGHT_ON": light.off(); lastAction = "LIGHT_OFF"; break;
            case "LIGHT_OFF": light.on(); lastAction = "LIGHT_ON"; break;
            case "AC_ON": ac.off(); lastAction = "AC_OFF"; break;
            case "AC_OFF": ac.on(); lastAction = "AC_ON"; break;
            default: System.out.println("No action to undo."); break;
        }
    }
}

// Client Code
public class Main {
    public static void main(String[] args) {
        Light light = new Light();
```

```
        AC ac = new AC();
        NaiveRemoteControl remote = new NaiveRemoteControl(light, ac);

        remote.pressLightOn();
        remote.pressACOn();
        remote.pressLightOff();
        remote.pressUndo(); // Should undo LIGHT_OFF -> Light ON
        remote.pressUndo(); // Should undo AC_ON -> AC OFF
    }
}

# Receiver classes - Light and AC with basic on/off methods
class Light:
    def on(self):
        print("Light turned ON")

    def off(self):
        print("Light turned OFF")

class AC:
    def on(self):
```

```
        print("AC turned ON")

    def off(self):
        print("AC turned OFF")

# Invoker - NaiveRemoteControl class to control devices
class NaiveRemoteControl:
    def __init__(self, light, ac):
        self.light = light
        self.ac = ac
        self.lastAction = ""

    # Command methods
    def pressLightOn(self):
        self.light.on()
        self.lastAction = "LIGHT_ON"

    def pressLightOff(self):
        self.light.off()
        self.lastAction = "LIGHT_OFF"
```

```
def pressACOn(self):
    self.ac.on()
    self.lastAction = "AC_ON"

def pressACOff(self):
    self.ac.off()
    self.lastAction = "AC_OFF"

# Undo last action
def pressUndo(self):
    if self.lastAction == "LIGHT_ON":
        self.light.off()
        self.lastAction = "LIGHT_OFF"
    elif self.lastAction == "LIGHT_OFF":
        self.light.on()
        self.lastAction = "LIGHT_ON"
    elif self.lastAction == "AC_ON":
        self.ac.off()
        self.lastAction = "AC_OFF"
    elif self.lastAction == "AC_OFF":
        self.ac.on()
```

```
            self.lastAction = "AC_ON"
        else:
            print("No action to undo.")

# Client Code
if __name__ == "__main__":
    light = Light()
    ac = AC()
    remote = NaiveRemoteControl(light, ac)

    remote.pressLightOn()
    remote.pressACOn()
    remote.pressLightOff()
    remote.pressUndo()  # Should undo LIGHT_OFF -> Light ON
    remote.pressUndo()  # Should undo AC_ON -> AC OFF
```

While the implementation works, it suffers from some significant issues.

Issues in the Code

- **1. Tight Coupling:**
  The `NaiveRemoteControl` class directly calls methods on the `Light` and `AC` classes. If additional devices need to be added in the future,

changes will be required in the remote control class. This violates the open/closed principle, where classes should be open for extension but closed for modification.

- **2. Lack of Flexibility:**
  The commands are hardcoded in the remote control class. If new actions or different command sequences are required, modifying the remote control code is necessary, leading to potential maintenance challenges.
- **3. Undo Functionality:**
  The `pressUndo` method is tightly coupled with the commands. This makes it difficult to add more complex undo functionality, especially when dealing with multiple actions or a variety of devices.
- **4. Hardcoded Commands:**
  The remote control class directly defines commands like `pressLightOn`, `pressACOn`, etc. This makes the system rigid and difficult to modify. Adding new actions or commands would require changing the remote control code, leading to challenges in maintaining or extending the system.
- **5. Maintaining Command History:**
  The original approach doesn't have a centralized mechanism to track previously executed commands. This leads to difficulties in implementing features like undo, where the last action needs to be reversed efficiently.

The Solution

The issues in the previous implementation can be addressed by using the Command Pattern. By applying this pattern, it becomes easier to encapsulate requests as objects, allowing for flexible and reusable command handling. The command pattern decouples the request sender (Invoker) from the receiver (Light/AC) and provides a unified way to handle multiple commands and actions.

Code Implementation:

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;

// ========= Receiver classes ===========
// Light and AC with basic on/off methods
class Light {
public:
    void on() {
        cout << "Light turned ON" << endl;
    }

    void off() {
        cout << "Light turned OFF" << endl;
    }
};

class AC {
public:
```

```
    void on() {
        cout << "AC turned ON" << endl;
    }

    void off() {
        cout << "AC turned OFF" << endl;
    }
};

// ========== Command interface ===========
// defines the command structure
class Command {
public:
    virtual void execute() = 0;
    virtual void undo() = 0;
};

// Concrete commands for Light ON and OFF
class LightOnCommand : public Command {
private:
    Light* light;
```

```
public:
    LightOnCommand(Light* light) {
        this->light = light;
    }

    void execute() override {
        light->on();
    }

    void undo() override {
        light->off();
    }
};

class LightOffCommand : public Command {
private:
    Light* light;

public:
    LightOffCommand(Light* light) {
```

```
    this->light = light;
  }

  void execute() override {
    light->off();
  }

  void undo() override {
    light->on();
  }
};

// Concrete commands for AC ON and OFF
class AConCommand : public Command {
private:
  AC* ac;

public:
  AConCommand(AC* ac) {
    this->ac = ac;
  }
```

```
    void execute() override {
        ac->on();
    }

    void undo() override {
        ac->off();
    }
};

class ACOffCommand : public Command {
private:
    AC* ac;

public:
    ACOffCommand(AC* ac) {
        this->ac = ac;
    }

    void execute() override {
        ac->off();
```

```
    }

    void undo() override {
        ac->on();
    }
};

// ========== Remote control class (Invoker) ==========
class RemoteControl {
private:
    Command* buttons[4];  // Assigning 4 slots for commands
    stack<Command*> commandHistory;

public:
    // Assign command to slot
    void setCommand(int slot, Command* command) {
        buttons[slot] = command;
    }

    // Press the button to execute the command
    void pressButton(int slot) {
```

```
        if (buttons[slot] != nullptr) {
            buttons[slot]->execute();
            commandHistory.push(buttons[slot]);
        } else {
            cout << "No command assigned to slot " << slot << endl;
        }
    }

    // Undo the last action
    void pressUndo() {
        if (!commandHistory.empty()) {
            commandHistory.top()->undo();
            commandHistory.pop();
        } else {
            cout << "No commands to undo." << endl;
        }
    }
};

// ========== Client code ===========
int main() {
```

```
Light light;
AC ac;

Command* lightOn = new LightOnCommand(&light);
Command* lightOff = new LightOffCommand(&light);
Command* acOn = new AConCommand(&ac);
Command* acOff = new ACOffCommand(&ac);

RemoteControl remote;
remote.setCommand(0, lightOn);
remote.setCommand(1, lightOff);
remote.setCommand(2, acOn);
remote.setCommand(3, acOff);

remote.pressButton(0); // Light ON
remote.pressButton(2); // AC ON
remote.pressButton(1); // Light OFF
remote.pressUndo();    // Undo Light OFF -> Light ON
remote.pressUndo();    // Undo AC ON -> AC OFF

return 0;
```

```
}

import java.util.*;

// ========= Receiver classes ==========
// Light and AC with basic on/off methods
class Light {
    public void on() {
        System.out.println("Light turned ON");
    }

    public void off() {
        System.out.println("Light turned OFF");
    }
}

class AC {
    public void on() {
        System.out.println("AC turned ON");
    }
```

```
    public void off() {
        System.out.println("AC turned OFF");
    }
}

// ========= Command interface ==========
//    defines the command structure
interface Command {
    void execute();
    void undo();
}

// Concrete commands for Light ON and OFF
class LightOnCommand implements Command {
    private Light light;

    public LightOnCommand(Light light) {
        this.light = light;
    }

    public void execute() {
```

```
        light.on();
    }

    public void undo() {
        light.off();
    }
}

class LightOffCommand implements Command {
    private Light light;

    public LightOffCommand(Light light) {
        this.light = light;
    }

    public void execute() {
        light.off();
    }

    public void undo() {
        light.on();
```

```
    }
}

// Concrete commands for AC ON and OFF
class AConCommand implements Command {
    private AC ac;

    public AConCommand(AC ac) {
        this.ac = ac;
    }

    public void execute() {
        ac.on();
    }

    public void undo() {
        ac.off();
    }
}

class ACOffCommand implements Command {
```

```
    private AC ac;

    public ACOffCommand(AC ac) {
        this.ac = ac;
    }

    public void execute() {
        ac.off();
    }

    public void undo() {
        ac.on();
    }
}

// ========== Remote control class (Invoker) ==========
class RemoteControl {
    private Command[] buttons = new Command[4];  // Assigning 4 slots for commands
    private Stack<Command> commandHistory = new Stack<>();

    // Assign command to slot
```

```
public void setCommand(int slot, Command command) {
    buttons[slot] = command;
}

// Press the button to execute the command
public void pressButton(int slot) {
    if (buttons[slot] != null) {
        buttons[slot].execute();
        commandHistory.push(buttons[slot]);
    } else {
        System.out.println("No command assigned to slot " + slot);
    }
}

// Undo the last action
public void pressUndo() {
    if (!commandHistory.isEmpty()) {
        commandHistory.pop().undo();
    } else {
        System.out.println("No commands to undo.");
    }
```

```
    }
}

// ========== Client code ===========
public class Main {
    public static void main(String[] args) {
        Light light = new Light();
        AC ac = new AC();

        Command lightOn = new LightOnCommand(light);
        Command lightOff = new LightOffCommand(light);
        Command acOn = new AConCommand(ac);
        Command acOff = new ACOffCommand(ac);

        RemoteControl remote = new RemoteControl();
        remote.setCommand(0, lightOn);
        remote.setCommand(1, lightOff);
        remote.setCommand(2, acOn);
        remote.setCommand(3, acOff);

        remote.pressButton(0); // Light ON
```

```
        remote.pressButton(2); // AC ON
        remote.pressButton(1); // Light OFF
        remote.pressUndo();    // Undo Light OFF -> Light ON
        remote.pressUndo();    // Undo AC ON -> AC OFF
    }
}

# ========= Receiver classes ==========
# Light and AC with basic on/off methods
class Light:
    def on(self):
        print("Light turned ON")

    def off(self):
        print("Light turned OFF")

class AC:
    def on(self):
        print("AC turned ON")

    def off(self):
```

```
        print("AC turned OFF")


# ========== Command interface ===========
# defines the command structure
class Command:
    def execute(self):
        pass

    def undo(self):
        pass

# Concrete commands for Light ON and OFF
class LightOnCommand(Command):
    def __init__(self, light):
        self.light = light

    def execute(self):
        self.light.on()

    def undo(self):
        self.light.off()
```

```
class LightOffCommand(Command):
    def __init__(self, light):
        self.light = light

    def execute(self):
        self.light.off()

    def undo(self):
        self.light.on()

# Concrete commands for AC ON and OFF
class AConCommand(Command):
    def __init__(self, ac):
        self.ac = ac

    def execute(self):
        self.ac.on()

    def undo(self):
        self.ac.off()
```

```
class ACOffCommand(Command):
    def __init__(self, ac):
        self.ac = ac

    def execute(self):
        self.ac.off()

    def undo(self):
        self.ac.on()

# ========== Remote control class (Invoker) ==========
class RemoteControl:
    def __init__(self):
        self.buttons = [None] * 4  # Assigning 4 slots for commands
        self.commandHistory = []

    # Assign command to slot
    def setCommand(self, slot, command):
        self.buttons[slot] = command
```

```
    # Press the button to execute the command
    def pressButton(self, slot):
        if self.buttons[slot] is not None:
            self.buttons[slot].execute()
            self.commandHistory.append(self.buttons[slot])
        else:
            print(f"No command assigned to slot {slot}")

    # Undo the last action
    def pressUndo(self):
        if self.commandHistory:
            self.commandHistory.pop().undo()
        else:
            print("No commands to undo.")

# ========== Client code ===========
if __name__ == "__main__":
    light = Light()
    ac = AC()

    lightOn = LightOnCommand(light)
```

```
lightOff = LightOffCommand(light)

acOn = AConCommand(ac)

acOff = ACOffCommand(ac)


remote = RemoteControl()

remote.setCommand(0, lightOn)

remote.setCommand(1, lightOff)

remote.setCommand(2, acOn)

remote.setCommand(3, acOff)


remote.pressButton(0)  # Light ON

remote.pressButton(2)  # AC ON

remote.pressButton(1)  # Light OFF

remote.pressUndo()     # Undo Light OFF -> Light ON

remote.pressUndo()     # Undo AC ON -> AC OFF
```

Let's now understand how the **Command Pattern** resolves the above discussed issues:

Issue	How Command Pattern Resolves the Issue
Tight Coupling	By using the Command Pattern, the `RemoteControl` class no longer directly interacts with the devices. It now interacts with command objects (e.g., `LightOnCommand`, `ACOffCommand`), which decouples the logic.
Lack of Flexibility	With the Command Pattern, new commands (e.g., for new devices or actions) can be created as new **`Command`** implementations without changing the **`RemoteControl`** class. This allows for easy extension.
Undo Functionality	The Command Pattern provides a consistent structure for undoing commands. Each concrete command (e.g., **`LightOnCommand`**, **`ACOffCommand`**) has its own **`undo()`** method, which allows easy reversal of actions.
Hardcoded Commands	The Command Pattern uses an interface for commands, which allows dynamic assignment of different commands to slots in the remote. This makes the command assignments flexible and customizable.
Maintaining Command History	The Command Pattern introduces a stack (**`commandHistory`**) in the **`RemoteControl`** class, which tracks previously executed commands. This makes the undo functionality centralized and easier to manage.

Impact Without the Command Pattern

- **Tight Coupling Between Invoker and Receiver**
  The invoker and receiver are directly linked, making future changes or additions to the system difficult without modifying both components.
- **Lack of Reusability**
  No abstraction for actions limits the ability to reuse code for different functionalities or scenarios across various parts of the application.
- **Undo/Redo Operations Not Supported**
  Implementing undo or redo functionality becomes complex and error-prone when operations are directly tied to specific actions.
- **Difficulty in Implementing Batch Actions**
  Implementing batch operations, like night mode changes, becomes cumbersome as each action needs to be handled individually.
- **No Plug-and-Play Flexibility**
  The system lacks the flexibility to add or modify commands dynamically without impacting other parts of the application.
- **Scalability Issues**
  As the system grows, managing commands and handling new features becomes increasingly difficult without a structured approach like the Command Pattern.

When to Use the Command Pattern

- **Decoupling Sender from Receiver**
  Use the Command Pattern when there is a need to decouple the sender (Invoker) from the receiver (the object performing the action).
- **Undo/Redo Support**
  The Command Pattern is useful when you require built-in support for undoing or redoing actions.

- **Batch Operations**
  When multiple actions need to be executed as part of a batch (e.g., applying night mode), the Command Pattern allows easy implementation.
- **Plug-in Architecture**
  It facilitates the creation of flexible, extensible systems where new commands can be added without affecting the core system.
- **Creating Macros or Composite Commands**
  Use the pattern to group multiple commands together, enabling complex actions to be executed in sequence as a single macro.

Pros and Cons of the Command Pattern

Pros

- **Decouples Sender and Receiver**
  The sender (Invoker) and receiver (the device or action) are decoupled, allowing for flexibility and easier maintenance.
- **Supports Undo/Redo Functionality**
  The Command Pattern inherently supports undo and redo actions, allowing for easier management of state reversals.
- **Easily Extensible and Reusable**
  New commands can be added without modifying existing code, and commands can be reused across different parts of the application.

Cons

- **Increases the Number of Classes**
  Implementing the Command Pattern can result in a large number

of small classes for each command, potentially increasing the complexity.

- **Can Add Unnecessary Complexity for Simple Tasks**
  For simple applications, the Command Pattern may introduce unnecessary complexity, making it harder to manage than simpler alternatives.
- **Requires Careful Design for Undo/Redo**
  Implementing undo/redo functionality correctly requires careful design and additional effort, especially for complex command chains.

Class Diagram